

Project Executable Files

Date	07 July 2026
Team ID	—
Project Name	Credit Card Approval Prediction
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

CreditCardApprovalPrediction/

```

├── Dataset/
│   ├── clean_dataset.csv      # Raw credit card dataset (CSV format)
│   └── clean_dataset.xlsx     # Raw credit card dataset (Excel format)
├── Flask/
│   ├── app.py                 # Core Flask application & /predict endpoint
│   ├── card_model.joblib      # Trained and serialized XGBoost Classifier
│   ├── encoders.joblib        # Label encoders for categorical features
│   ├── feature_cols.joblib    # Feature column order for inference
│   ├── templates/             # HTML structure for the web interface
│   │   └── index.html         # Application form & prediction result page
│   └── static/                 # Static visual assets and client-side logic
│       ├── css/
│       │   └── style.css      # Custom stylesheet (dark theme, blue gradient)
│       └── js/
│           └── script.js      # Async form submission & result rendering
├── Training/
│   └── Credit_Card_Approval.ipynb # Notebook: EDA, preprocessing, model
comparison
├── requirements.txt            # Required Python package dependencies
└── runtime.txt                 # Target Python runtime spec (python-3.11.9)
  
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://credit-card-approval-prediction-2-mfxo.onrender.com/
Login Credentials (Demo)	NA (Publicly accessible interface; does not require administrative login credentials to request a prediction)
Platform / Hosting Provider	Render
Repository Link	https://github.com/manojmulammagari/Credit-Card-Approval-Prediction
Demo Video Link	https://drive.google.com/file/d/1tPefa51gM4BXCvQj9YrcYmN6ms-ZfDf_/view?usp=drive_link

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Follow these step-by-step instructions to set up and run the application in a local development environment:

Prerequisites

Python 3.8+ installed (recommended version: Python 3.11.9)

Visual Studio Code or command-line terminal access

Local Running Procedure

Extract and Navigate to Project Directory:

```
cd CreditCardApprovalPrediction
```

Initialize a Virtual Environment (Optional but Recommended):

```
python -m venv venv
```

```
# Activate on Windows:
```

```
venv\Scripts\activate
```

```
# Activate on macOS/Linux:
```

```
source venv/bin/activate
```

Install Dependencies: Install requirements using the project package file:

```
pip install -r requirements.txt
```

Verify / Recreate Trained Model (Optional): Pre-trained serialized model and encoders are pre-packaged in the workspace. To train the models from scratch:

Open Training/Credit_Card_Approval.ipynb in Jupyter Notebook/VS Code and execute all cells. This exports the trained XGBoost model as card_model.joblib and the label encoders as encoders.joblib into the /Flask folder.

Start Flask Server: Navigate into the web application folder and run the boot script:

```
cd Flask
```

```
python app.py
```

Open in Browser: Open your browser and navigate to:

```
http://127.0.0.1:5000
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Internal exception details shown to users (information leak).	Return generic user-facing errors and log full details server-side; avoid rendering raw exception text.
2	No Database Persistence: prediction results and inputs are processed on the fly and are not stored in any back-end database.	Workaround: Reloading the app resets the form. Acceptable for an academic demo; a database can be added for production use.
3	Input Out-of-Bounds Scaling: Supplying extreme inputs (e.g. very high income) outside the training dataset distribution may distort prediction confidence.	Workaround: Use realistic applicant scenarios within the training data range (income up to ~₹150,000).